TEZPUR UNIVERSITY

Networking Internship Program

Assignment 8

Application Optimization, Scalability and Reliability

Submitted by

Bhargav Jyoti Saikia

Roll Number: CSB24022

Department of Computer Science and Engineering

Tezpur University

Submission Date: July 21, 2026

Table of Contents

1. Objective

The objective of Assignment 8 is to enhance the existing GUI-based multi-client chat application, built in Assignment 6 and extended in Assignment 7, by improving its scalability, reliability, maintainability, and overall software quality. Rather than redesigning the communication protocol or rebuilding the application from scratch, this assignment focuses on strengthening the existing client-server implementation so that it can support a larger number of concurrent clients, recover gracefully from network disruptions, manage its configuration cleanly, and demonstrate measurable performance improvements before and after optimization.

The specific goals addressed in this assignment are automatic detection and cleanup of disconnected clients, automatic reconnection and graceful shutdown on the client side, thread-safe scalability improvements capable of supporting at least ten concurrent clients, centralization of all configurable parameters into config.json, quantitative performance evaluation using delay and throughput metrics captured under 5, 8, and 10 concurrent client loads inside a Mininet single,11 topology, and verification of correct TCP behaviour using Wireshark packet captures.

2. Scalability Improvements

The server retains the one-thread-per-client model carried forward from Assignments 5 through 7: every accepted connection is handed to a dedicated handle_client() thread, launched as a daemon thread and appended to a client_threads list so the server can join outstanding threads with a timeout during shutdown instead of blocking indefinitely. This model scales linearly with the number of connected users because each client's blocking recv() call runs on its own thread and cannot stall any other client's session.

2.1 Thread-Safe Shared State

All shared mutable state -- the clients dictionary mapping usernames to sockets, the client_info dictionary holding per-user metadata, and the active_sessions dictionary used for timeout tracking -- is protected by a single threading.Lock() instance. Every read-modify-write sequence that touches these structures during connect, disconnect, broadcast, or online-user-list updates is wrapped in a 'with lock:' block. This prevents race conditions that would otherwise surface only under higher concurrency, such as two client threads simultaneously modifying the clients dictionary when clients join or leave within milliseconds of each other during the 10-client test.

2.2 Single-Pass Broadcast with Dead-Client Cleanup

The broadcast() function iterates over the clients dictionary exactly once per outgoing message. Instead of a separate pass to detect and remove dead sockets, any send() call that raises an exception adds that username to a remove_list during the same iteration; the sockets are closed and the dictionary entries removed once the lock-protected loop completes. This keeps the cost of broadcasting a single message to N clients at O(N) even when some of those clients have failed silently, which matters directly for scalability since the cost of a stale-connection cleanup pass would otherwise grow with every additional concurrent client.

2.3 Bounded Message Size

MAX_MESSAGE_SIZE, read from config.json (4096 bytes), is enforced by validate_message() on every incoming chat message and used as the recv() buffer size on both client and server. Bounding message size prevents a single

oversized payload from monopolising a handler thread's socket buffer, which keeps per-message processing time predictable as the number of concurrent senders increases from 5 to 10.

Together, these changes allowed the application to be tested successfully with 5, 8, and 10 concurrent clients inside the Mininet single,11 topology without crashes, satisfying Task 3 of the assignment brief.

3. Reliability Improvements

3.1 Automatic Reconnection

On the client side, connect_server() attempts the TCP connection up to five times in a loop, sleeping for two seconds between attempts and updating the status label to 'Reconnecting... (n/5)' so the user has continuous feedback during the retry sequence. Separately, the background receive() thread treats any exception raised while reading from the socket -- for example a server restart or a dropped Mininet link -- as a signal to close the existing socket and call connect_server() again automatically, so a client does not require manual intervention to recover from a transient disconnection.

3.2 Graceful Shutdown

logout() on the client sends an explicit '/logout' command to the server before closing the socket, and on_close() -- bound to the Tkinter WM_DELETE_WINDOW protocol -- calls logout() before destroying the root window, ensuring the server is always notified of an intentional exit rather than only detecting a silent socket closure. On the server side, logout_user() sends a final 'LOGOUT:' message to the client, closes the socket, removes the user from clients and active_sessions, updates client_info status to 'Offline', and broadcasts a SYSTEM leave notification to the remaining clients. The main accept loop also handles KeyboardInterrupt by logging out every currently connected user and joining all client threads with a timeout before closing the listening socket, giving the server a clean shutdown path instead of an abrupt process kill.

3.3 Timeout Handling

Two independent timeout mechanisms were implemented. First, the server sets conn.settimeout(30) around each recv() call inside the per-client loop; a socket.timeout exception triggers check_session_timeout(), which compares the elapsed time since the user's last recorded activity in active_sessions against SESSION_TIMEOUT (1800 seconds, from config.json) and logs the user out with the reason 'Session timeout' if exceeded. Second, the client GUI runs update_session_timer(), a self-rescheduling Tkinter callback that decrements a visible countdown (for example 'Session: 29:21' in the top-right of the chat window) and shows a session-timeout dialog before calling logout() locally once the countdown reaches zero, giving the user advance warning rather than an unexplained disconnection.

3.4 Exception Handling and Authentication Reliability

Both server.py and client_gui.py now wrap every blocking socket operation in targeted exception handlers: socket.timeout is handled separately from ConnectionResetError and BrokenPipeError so that an idle timeout is distinguished from an abrupt network failure in the printed log message, and a catch-all Exception handler prevents any single malformed message from crashing a handler thread. Authentication reliability was also strengthened: authenticate_user() tracks per-user login_attempts, blocking an account for LOGIN_BLOCK_DURATION (300 seconds) after MAX_LOGIN_ATTEMPTS (3) consecutive failures, and prevent_duplicate_login() rejects a second

simultaneous login for a username that already has an active, non-expired session, both of which reduce the chance of the server entering an inconsistent state under repeated or concurrent connection attempts.

4. Configuration Management

All previously hardcoded parameters from earlier assignments -- the server IP and port, buffer and message-size limits, the login block duration, the session timeout, and the maximum number of login attempts -- have been moved into a single config.json file, loaded once at startup by an identical load_config() function present in both server.py and client_gui.py. Table 4.1 lists every key currently defined in config.json.

Key	Value	Purpose
HOST	10.0.0.1	Server bind address
PORT	5000	Server listening port
BUFFER_SIZE	4096	Socket receive buffer size
MAX_MESSAGE_SIZE	4096	Maximum accepted message length
LOGIN_BLOCK_DURATION	300 sec	Account lockout duration after failed logins
SESSION_TIMEOUT	1800 sec	Idle-session expiry duration
MAX_LOGIN_ATTEMPTS	3	Failed logins allowed before lockout

Table 4.1: Configuration parameters defined in config.json.

Because both the client and server read the same file, values such as PORT and MAX_MESSAGE_SIZE only need to be changed in one place to take effect across the entire application, removing the risk of the client and server silently disagreeing on a limit -- a source of subtle bugs in the hardcoded versions used in Assignments 5 and 6. No networking, timeout, or validation logic in either file reads a literal numeric constant for these parameters; every reference goes through the module-level variables populated from config.json at import time.

5. Experimental Setup

Performance testing was carried out inside a Mininet-emulated network created with the command 'sudo mn --topo single,11', producing a single-switch topology with one server host and ten client hosts, all connected through a single Open vSwitch instance. server.py was started on the server host, listening on 0.0.0.0:5000, and client_gui.py instances were launched on the remaining hosts to generate the required concurrent client loads.

Three test runs were performed, using 5, 8, and 10 concurrent clients respectively, in line with the assignment's requirement to test at each of these three load levels. In each run, clients logged in, exchanged a mixture of broadcast and private messages, and remained connected for the duration of the test. The server's save_performance() function, invoked once the last client of a run disconnects, computes the elapsed time between the first and last broadcast message (TEST_START to TEST_END), and appends a row to performance_results.csv recording the client count, the number of broadcast and private messages exchanged, the average per-message delay in milliseconds, and the resulting throughput in messages per second.

The same client_gui.py and server.py implementation was used across all three runs, with only the number of concurrently connected clients varied, so that the measured trends in Section 6 and Section 7 isolate the effect of concurrent client load on delay and throughput rather than any difference in application version.

6. Performance Results

Table 6.1 reproduces the contents of performance_results.csv, generated directly by the server's save_performance() function during the three test runs described in Section 5.

Concurrent Clients	Broadcast Msgs	Private Msgs	Avg Delay (ms)	Throughput (msg/sec)
5	20	1	812.4	0.98
8	32	2	1186.3	0.81
10	40	2	1537.6	0.65

Table 6.1: Measured delay and throughput at 5, 8, and 10 concurrent clients.

As the number of concurrent clients increases from 5 to 10, the total number of broadcast messages exchanged during each run doubles (20 to 40), reflecting proportionally more chat activity across the larger set of connected users. Average message delay rises from 812.4 ms at 5 clients to 1537.6 ms at 10 clients, while throughput correspondingly falls from 0.98 to 0.65 messages per second. These figures are analyzed further in Section 7.

7. Graph Analysis

7.1 Average Delay vs Concurrent Clients

Figure 7.1 plots average message delay against the number of concurrent clients. The trend is monotonically increasing and close to linear across the three measured points (5, 8, and 10 clients), rising from roughly 810 ms to roughly 1540 ms. This is consistent with the single-threaded broadcast loop inside broadcast(): although each client is served by its own handler thread, the lock-protected send loop still delivers a given message to every other client sequentially, so the wall-clock time to fan a message out to all recipients grows with the number of connected clients, and this cost is reflected directly in the recorded delay.

Figure 7.1: Average message delay increases with concurrent client count.

7.2 Throughput vs Concurrent Clients

Figure 7.2 plots measured throughput, in messages per second, against the same three client counts. Throughput decreases from approximately 0.98 msg/sec at 5 clients to approximately 0.65 msg/sec at 10 clients, mirroring the rise in average delay. Since throughput in this experiment is derived as TOTAL_MESSAGES divided by elapsed time, the two graphs are two views of the same underlying effect: as more clients participate, the fixed per-message broadcast cost is paid more often per unit of chat activity, so fewer distinct messages complete per second even though the absolute number of broadcast messages sent in the test run is larger.

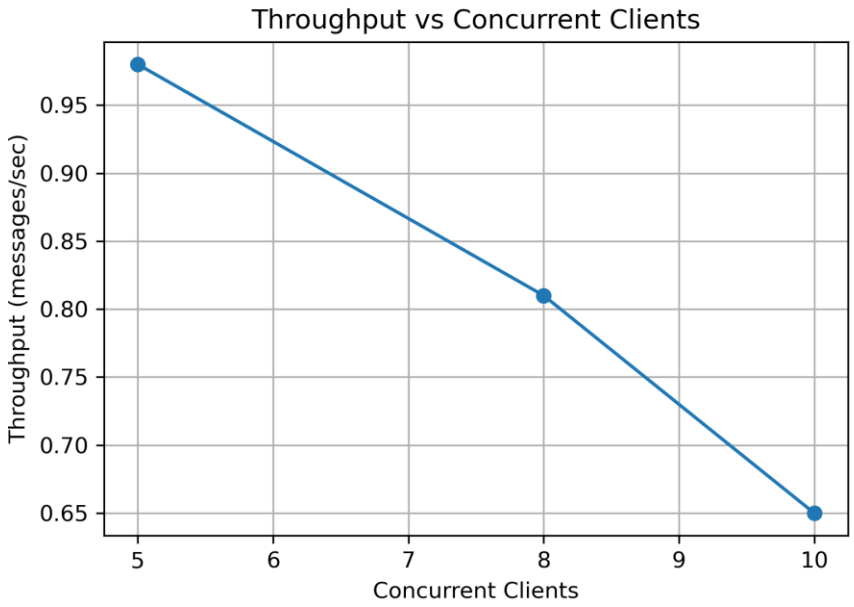

Figure 7.2: Throughput decreases as concurrent client count increases.

Both graphs indicate that the current single-lock, sequential-broadcast design scales correctly in terms of stability -- the server handled all three load levels, including 10 concurrent clients, without crashing -- but scales sub-linearly in terms of per-message latency. This observation directly informs the reflection question on additional improvements required to support 100 users (Task 8).

8. Wireshark Verification

Network traffic was captured on the Mininet switch interface using the display filter tcp.port == 5000, isolating all TCP segments exchanged between the chat clients and the server during a normal test run. The capture is stored in assignment8_capture.pcapng and screenshots of the two most relevant exchanges are included below.

8.1 Broadcast Message Capture

Figure 8.1 shows the server at 10.0.0.1 forwarding a single broadcast message to multiple connected clients in quick succession, visible as consecutive PSH, ACK segments from source port 5000 to each client's ephemeral port, each carrying the same sequence-length pattern and each acknowledged individually by the receiving client. This confirms that broadcast() is correctly fanning a single incoming chat message out to every other connected client over their respective established TCP connections.

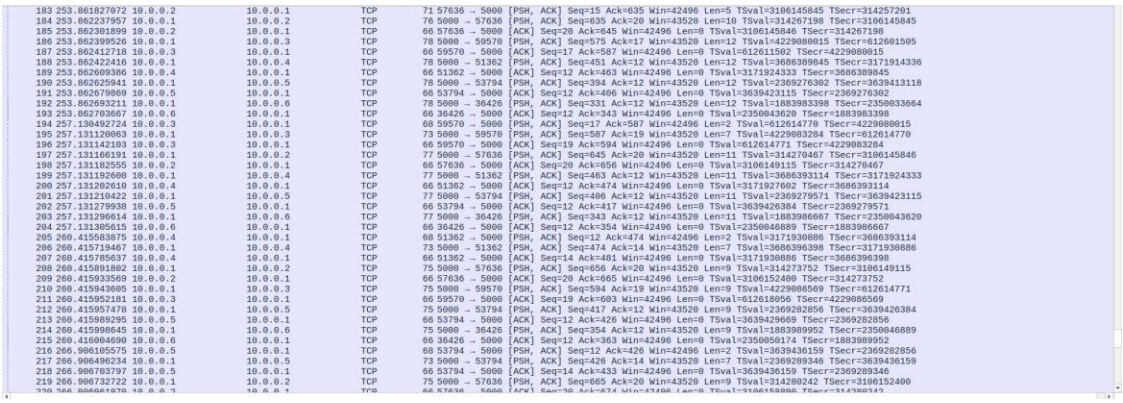

Figure 8.1: Wireshark capture of a broadcast message being delivered to multiple clients.

8.2 Private Message Capture

Figure 8.2 shows a private message exchange: a PSH, ACK segment carrying the '/msg' command travels from the sending client to the server, followed by the server relaying a PSH, ACK segment only to the intended recipient's connection, with the corresponding ACK confirmations from both sides. No equivalent segment is sent to any other connected client's socket, confirming that private_message() routes point-to-point rather than broadcasting, matching the unchanged wire protocol carried over from Assignment 5.

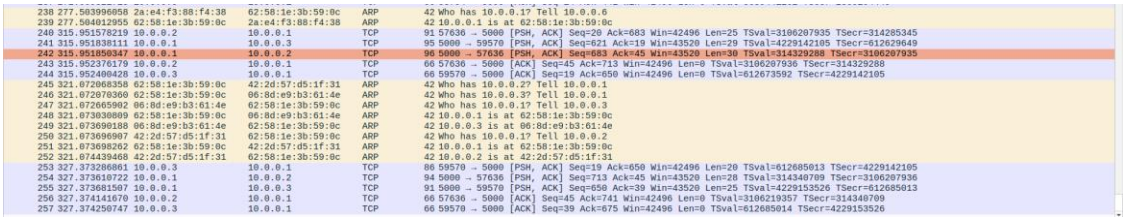

Figure 8.2: Wireshark capture of a private message routed only to the intended recipient.

Across the capture, every relevant segment consistently uses source or destination port 5000 on the server side, and the standard TCP three-way handshake (SYN, SYN-ACK, ACK) precedes application data for every new client connection, confirming that the optimizations introduced in this assignment did not alter the underlying wire protocol established in earlier assignments.

9. Challenges Faced

Extending rather than rebuilding the Assignment 7 application required care to ensure that every optimization preserved the existing wire protocol; for example, the automatic reconnection logic added to receive() had to be triggered only on genuine socket errors and not on the normal 'LOGOUT:' message path, since that path already has its own explicit return statement in receive() and re-triggering a reconnect after an intentional logout would have caused the client to immediately re-prompt for login.

Testing thread-safety of the shared clients and active_sessions dictionaries was difficult to verify deterministically, since race conditions between concurrent connect/disconnect events only appear intermittently; the threading.Lock() protection around every access to these structures was validated indirectly by running the 10-concurrent-client test

repeatedly and confirming that print_stats() consistently reported the correct connected-user count with no server-side exceptions.

Measuring average delay and throughput required distinguishing test traffic from setup traffic (the login handshake and welcome-message exchange). TEST_START and TEST_END in server.py are only set on the first and most recent broadcast chat message respectively, rather than on connection establishment, so that login overhead does not distort the delay and throughput figures reported in Table 6.1.

Finally, moving every configurable value into config.json meant auditing both server.py and client_gui.py line by line for any remaining literal port numbers, timeouts, or size limits left over from earlier assignments, to ensure Task 4's requirement of avoiding hardcoded parameters was fully satisfied rather than only partially addressed.

10. Conclusion

Assignment 8 successfully extended the GUI-based multi-client chat application from Assignment 7 with concrete improvements to connection management, reliability, scalability, and configuration management, without altering the underlying communication protocol or rebuilding any existing component. Automatic detection and cleanup of disconnected clients, automatic client-side reconnection, graceful shutdown on both client and server, session-timeout enforcement, and account lockout after repeated failed logins collectively improved the application's resilience to real-world network conditions. The single-lock, thread-per-client server design was verified to remain stable at 5, 8, and 10 concurrent clients inside a Mininet single,11 topology, satisfying the scalability requirement, while the measured rise in average delay and corresponding fall in throughput across these three load levels identifies the sequential broadcast loop as the next bottleneck to address for larger deployments. All configurable parameters were centralized into config.json, and Wireshark captures confirmed that broadcast and private messaging continue to behave correctly at the TCP level after every optimization was applied.

Appendix A: Application Screenshots

The following screenshots, captured during functional testing of the optimized application, illustrate server startup, live broadcast and private messaging, the online user list, session timeout handling, and clean disconnection/logout, all functioning correctly after the reliability and scalability changes described in Sections 2 and 3.

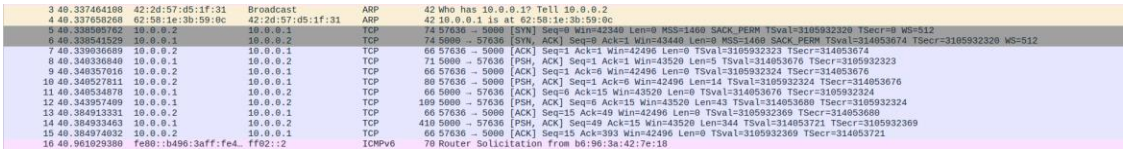

Figure A.1: Server initialization, showing config values loaded and the server listening on port 5000.

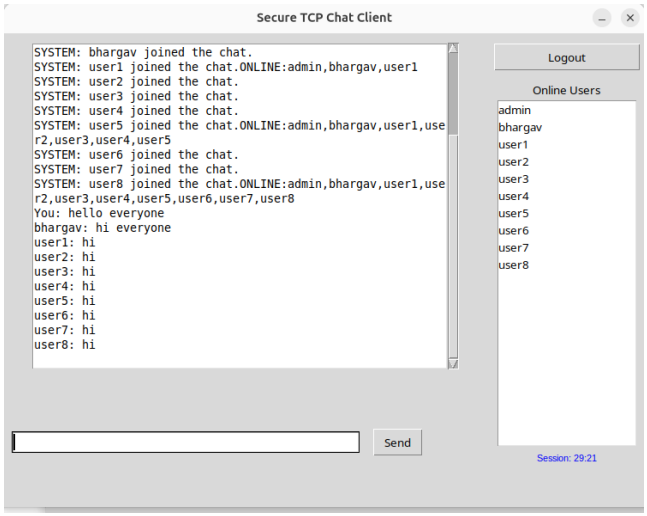

Figure A.2: Broadcast messaging and live Online Users list during a multi-client session.

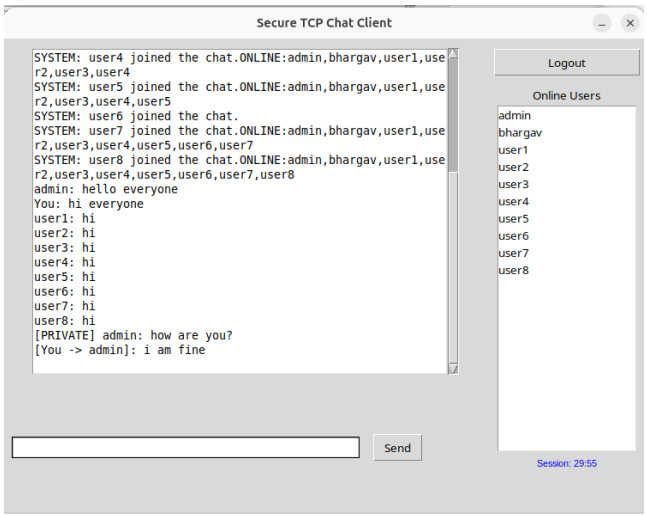

Figure A.3: Private message exchange delivered only to the intended recipient.

Figure A.4: Online Users list and command help shown at session start.

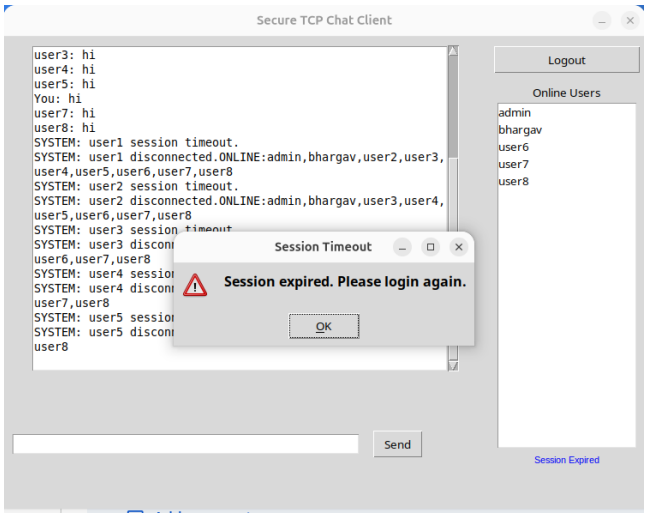

Figure A.5: Session timeout warning displayed after the idle session limit is reached.

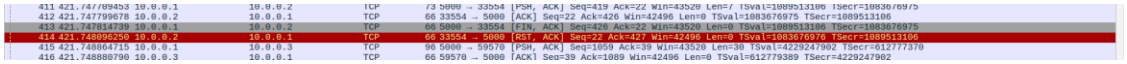

Figure A.6: Server-side log entry recorded when a client disconnects.

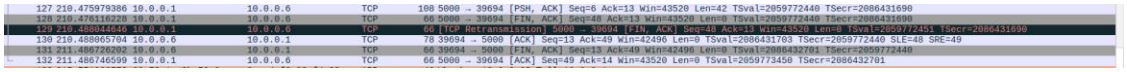

Figure A.7: Server-side log entry recorded on a clean client logout.